3

Tipos de dados

Universidade Estadual da Paraíba (UEPB)
Paradigmas de Programação
Kláudio Henrique Mascarenhas Medeiros

Tipos de dados

- Um tipo representa um conjunto de valores com comportamento uniforme sob operações que podem ser feitas entre esses valores.

Tipos de dados

- Um **tipo** representa um conjunto de valores com comportamento uniforme sob operações que podem ser feitas entre esses valores.

- Cada linguagem irá fornecer um **conjunto de tipos de dados** a serem utilizados.

Tipos de dados

- Um **tipo** representa um conjunto de valores com comportamento uniforme sob operações que podem ser feitas entre esses valores.

- Cada linguagem irá fornecer um **conjunto de tipos de dados** a serem utilizados.

- Algumas linguagens possuem tipos que não existem em outras (e.g. C tem ponteiros para variáveis, o que não existe em toda linguagem).

Tipos de dados

Quais os **tipos existentes** em C?

Tipos de dados

Exemplo: tipo boolean em várias linguagens

```
boolean flag1 = true;
boolean flag2 = false;
boolean flag3 = flag1 and flag2;
```

- Uma variável booleana pode assumir o valor verdadeiro (true) ou falso (false)
- Suporta operações como and, or e not.

Tipos de dados

Algumas linguagens oferecem operadores lógicos compostos

Tipos de dados

- C simula booleanos utilizando inteiros.
  - 0 é entendido como “falso”
  - outros valores são entendidos como “verdadeiro”
  - normalmente, utiliza-se 1 para “verdadeiro”

```c
#include <stdio.h>

int main()
{
    int n1 = 12;
    int n2 = 6;
    printf("%d\n", n1 > n2);
}
```

- C99 e C++ possuem o tipo booleano.

Tipos de dados

- C simula booleanos utilizando inteiros.

```c
#include <stdio.h>

int main()
{
    int cond = 1;
    if(cond)
        printf("Trecho executado!\n");
}
```

Tipos de dados

Lembrando que podemos criar nossos próprios tipos, mas isso é assunto para mais adiante

Tipos de dados

- O sistema de tipos de uma linguagem ajuda na detecção de erros nessa linguagem.

Tipos de dados

- O sistema de tipos de uma linguagem ajuda na **detecção de erros** nessa linguagem.

- O sistema de tipos também ajuda na **documentação** dessa linguagem, informando quais são os seus dados e dando alguns indícios de como funcionam.

Tipos de dados

- O sistema de tipos de uma linguagem ajuda na **detecção de erros** nessa linguagem.

- O sistema de tipos também ajuda na **documentação** dessa linguagem, informando quais são os seus dados e dando alguns indícios de como funcionam.

- Os tipos podem ser verificados em diferentes módulos, promovendo uma consistência da interface entre eles. Isso ajuda na **modularização dos programas**.

Tipos de dados

- O tipo de dado T pode ter uma determinada cardinalidade, indicada por #T, como sendo o número de possibilidades de valores diferentes para esses tipos.

Tipos de dados

Qual a **cardinalidade** do tipo boolean?

Tipos de dados

- O tipo de dado T pode ter uma determinada **cardinalidade**, indicada por #T, como sendo o número de possibilidades de valores diferentes para esses tipos.

- #Boolean é 2, #Character pode ser 128 (ASCII) ou 65536 (Unicode).

Tipos primitivos

- Um **tipo primitivo** em uma linguagem de programação é um tipo cujos valores não podem ser decompostos em valores mais simples.

Tipos primitivos

- Um **tipo primitivo** em uma linguagem de programação é um tipo cujos valores não podem ser decompostos em valores mais simples.

- A própria linguagem de programação já define quais são esses tipos primitivos.

Tipos primitivos

- Um **tipo primitivo** em uma linguagem de programação é um tipo cujos valores não podem ser decompostos em valores mais simples.

- A própria linguagem de programação já define quais são esses tipos primitivos.

- São exemplos int, bool, char, enum, etc. Em algumas linguagens, string pode ser considerado tipo primitivo (e.g. Python) e em outras não (e.g. Java).

Tipos primitivos

*"In addition to the eight primitive data types listed above, the Java programming language also provides special support for character strings via the java.lang.String class. Enclosing your character string within double quotes will automatically create a new String object; for example, String s = "this is a string";. String objects are immutable, which means that once created, their values cannot be changed. **The String class is not technically a primitive data type**, but considering the special support given to it by the language, **you'll probably tend to think of it as such**."*

Link: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

Tipos primitivos

Class String

java.lang.Object
    java.lang.String

All Implemented Interfaces:

Serializable, CharSequence, Comparable<String>

```
public final class String
extends Object
implements Serializable, Comparable<String>, CharSequence
```

The `String` class represents character strings. All string literals in Java programs, such as `"abc"`, are implemented as instances of this class.

Strings are constant; their values cannot be changed after they are created. String buffers support mutable strings. Because String objects are immutable they can be shared. For example:

```
String str = "abc";
```

is equivalent to:

```
char data[] = {'a', 'b', 'c'};
String str = new String(data);
```

Lembrete: em Java, uma classe final é uma classe que não pode ter subclasses. É como se a implementação dela não precisasse ser estendida (extends), fazendo-a ser definitiva (final).

Link: https://docs.oracle.com/javase/7/docs/api/java/lang/String.html

Tipos primitivos numéricos

- Os tipos primitivos numéricos são dos mais variados, como inteiros (e.g. int, long) ou de ponto flutuante (e.g. float, double).

Tipos primitivos numéricos

- Os **tipos primitivos numéricos** são dos mais variados, como inteiros (e.g. int, long) ou de ponto flutuante (e.g. float, double).

- A diferença entre eles está, por exemplo, na quantidade de bits disponíveis para armazená-los.
    - int e float têm 32 bits, long e double têm 64 bits

Tipos de dados

Em Rust, quando definimos uma variável numérica, podemos definir quantos bits queremos (e.g. f32, f64)

Tipos primitivos (novos)

- Certas linguagens permitem a definição de novos tipos primitivos (exemplo abaixo em Ada).

```
type PrimitiveType is range 0..100;
```

Tipos primitivos (novos)

- Certas linguagens permitem a definição de novos tipos primitivos (exemplo abaixo em Ada).

```
type PrimitiveType is range 0..100;
```

- O conjunto de valores que esse tipo pode contemplar é {0, ..., 100}.

- A cardinalidade de PrimitiveType é 101.

Tipos enum

- Os **tipos de dados enumeráveis** possuem valores na forma de constantes simbólicas, enumeradas na definição do enumerador.

Tipos enum

- Os **tipos de dados enumeráveis** possuem valores na forma de constantes simbólicas, enumeradas na definição do enumerador.

- Essas constantes são preenchidas implicitamente através de atribuições de valores inteiros.

Tipos enum (C)

```c
#include <stdio.h>

enum cor {PRETO, BRANCO};
typedef enum cor Cor;

int main()
{
    Cor cor1 = PRETO;        // enum cor cor1
    Cor cor2 = BRANCO;
    if(cor1 == PRETO)
        printf("Temos como cor1 a cor preta\n");
}
```

O typedef foi colocado aqui apenas para não termos que utilizar “enum cor” dentro do main()

Tipos enum

- Os **tipos de enumeração** são interessantes por terem valores nomeados, o que facilita o reconhecimento deles e compreensão do código (SEGUNDA remete claramente a ideia da segunda-feira).

Tipos enum

- Os **tipos de enumeração** são interessantes por terem valores nomeados, o que facilita o reconhecimento deles e compreensão do código (SEGUNDA remete claramente a ideia da segunda-feira).

- Um outro ponto interessante é que os valores listados são os valores permitidos, de modo que valores fora da faixa de enumeração não podem ser atribuídos.

Tipos enum (subfaixa)

- Algumas linguagens, como Ada, possuem tipos denominados de tipos subfaixa

```
type Dias is (Seg, Ter, Qua, Qui, Sex, Sab, Dom);
subtype Weekdays is Days range Seg..Sex;
subtype Index is Integer range 1..100;
```

Tipos enum (subfaixa)

- Algumas linguagens, como Ada, possuem tipos denominados de tipos subfaixa

```
type Dias is (Seg, Ter, Qua, Qui, Sex, Sab, Dom);
subtype Weekdays is Days range Seg..Sex;
subtype Index is Integer range 1..100;
```

- Esses tipos evidenciam que valores são possíveis, e valores fora desses intervalos especificados são considerados um erro.

Strings (linguagens de programação)

- Strings possuem valores que são sequências de zero ou mais caracteres.

Strings (linguagens de programação)

- **Strings** possuem valores que são sequências de zero ou mais caracteres.

- Como mencionado de forma breve em slides anteriores, não há um consenso de se string é um tipo primitivo ou composto.

Strings (linguagens de programação)

- **Strings** possuem valores que são sequências de zero ou mais caracteres.

- Como mencionado de forma breve em slides anteriores, não há um consenso de se string é um tipo primitivo ou composto.

- Em Python, é um tipo primitivo. Em C, é um array de char com '\0' no final. Em Java, é um objeto da classe String.

Strings e seus tamanhos

- As strings de tamanho estático possuem tamanho definido na sua declaração (preenchidos com vazio).

Strings e seus tamanhos

- As **strings de tamanho estático** possuem tamanho definido na sua declaração (preenchidos com vazio).

- As **strings de tamanho dinâmico limitado** possuem tamanho variado até um tamanho máximo.

Strings e seus tamanhos

- As **strings de tamanho estático** possuem tamanho definido na sua declaração (preenchidos com vazio).

- As **strings de tamanho dinâmico limitado** possuem tamanho variado até um tamanho máximo.

- As **strings de tamanho dinâmico** possuem tamanho variado, sem limites. Exige alocação e desalocação dinâmica de armazenamento, mas nesse caso há muita flexibilidade.

Strings e seus tamanhos

- Em Python, as strings têm tamanho estático. As strings Python são imutáveis. O que podemos fazer é gerar novas strings a partir de strings existentes.

Strings e seus tamanhos

```
string = "klaus"
string[2] = "e"
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    string[2] = "e"
TypeError: 'str' object does not support item assignment
```

Strings e seus tamanhos

```
string = "klaus"
print(id(string))
1302658549360
string = "stella"
print(id(string))
1302658578992
```

O id() em Python retorna a “identidade” de um objeto. Perceba que a identidade de ambos são diferentes.

```python
string = "klaus"
string = "stella"
print(string)
stella
for letra in string:
    print(letra, id(letra))
```

Perceba que o id de ambas as ocorrências de letra “l” em string foram iguais.

```
s 140730694277872
t 140730694277928
e 140730694277088
l 140730694277480
l 140730694277480
a 140730694276864
```

Strings (Python)

```
>>> string1 = "klaus"
>>> string2 = "stella"
>>> len(string1)
5
>>> string1 > string2
False
>>> string1 + string2
'klausstella'
>>> string1[1:3]
'la'
>>> string1[1:]
'laus'
```

Em algumas linguagens de programação, podemos obter slices (fatias) de listas ou strings, como as observadas nesse exemplo.

string[1:3] nos dá uma slice que contém os elementos na posição começando de 1 até 3, não incluindo ele (“la”)

string[1:] nos dá uma slice que contém os elementos na posição 1 até o final da string (“laus”)

Strings e seus tamanhos

- Em Python, as strings têm tamanho estático. As strings Python são imutáveis. O que podemos fazer é gerar novas strings a partir de strings existentes.

- Em C, o tamanho de uma string pode variar, mas entre zero e um máximo estipulado.

exemplo_plp_string.cpp

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char nome1[10];
7      char nome2[];
8  }
```

Em Java, você pode definir uma variável como sendo int[] (array de inteiros) e definir o tamanho apenas na instanciação (new int[10];)

Compiler (2) | Resources | Compile Log | Debug | Find Results | Close

Line	Col	File	Message
		D:\Documents\Codigos_C\exemplo_plp_string.cpp	**In function 'int main()':**
7	13	D:\Documents\Codigos_C\exemplo_plp_string.cpp	[Error] storage size of 'nome2' isn't known

Strings e seus tamanhos

- Em Python, as strings têm tamanho estático. As strings Python são imutáveis. O que podemos fazer é gerar novas strings a partir de strings existentes.

- Em C, o tamanho de uma string pode variar, mas entre zero e um máximo estipulado.

- Em JavaScript, as strings possuem tamanho dinâmico, inclusive sem tamanho máximo, tendo a máxima flexibilidade mas com intensa alocação e liberação.

Strings (C)

- Em C, strings são **arrays de char**, finalizados com um caractere nulo ('\0').

Strings (C)

- Em C, strings são **arrays de char**, finalizados com um caractere nulo (‘\0’).

- A string “nome” é formada por “nome\0”.

Strings (C)

- Em C, strings são **arrays de char**, finalizados com um caractere nulo (‘\0’).

- A string “nome” é formada por “nome\0”.

- Existe uma biblioteca que possui operações com strings chamada string.h, com operações como contagem de tamanho (strlen), cópia de valores (strcpy) ou comparação (strcmp).

Tipos compostos

- Os **tipos compostos** são tipos cujos valores são compostos a partir de outros valores, geralmente de tipos mais simples (e.g. listas, tuplas).

Tipos compostos

- Os **tipos compostos** são tipos cujos valores são compostos a partir de outros valores, geralmente de tipos mais simples (e.g. listas, tuplas).

- Em C, alguns exemplos de tipos compostos são as estruturas (struct) e as uniões (unions).

Tipos compostos

- Os **tipos compostos** são tipos cujos valores são compostos a partir de outros valores, geralmente de tipos mais simples (e.g. listas, tuplas).

- Em C, alguns exemplos de tipos compostos são as estruturas (struct) e as uniões (unions).

- Em Java, nós podemos definir classes, que possuem atributos e comportamentos (métodos).

Tipos compostos (C)

- Em C, podemos definir uma estrutura (struct) e que variáveis ela contempla.

Tipos compostos (C)

- Em C, podemos definir uma estrutura (struct) e que variáveis ela contempla.

- Podemos acessar cada variável isoladamente a partir da identificação desta variável.

Tipos compostos (C)

- Em C, podemos definir uma estrutura (struct) e que variáveis ela contempla.

- Podemos acessar cada variável isoladamente a partir da identificação desta variável.

- Comumente, os endereços de memória utilizados para os campos de um registro são vizinhos (podendo, a depender dos tipos desses campos, ter tamanhos diferentes).

Tipos compostos (C)

```c
#include <stdio.h>

struct data{
    int dia, mes, ano;
};

int main()
{
    struct data hoje = {24, 2, 2023};
    printf("O dia de hoje: %d\n", hoje.dia);
}
```

Tipos compostos (C)

```c
#include <stdio.h>

struct aluno{
    char nome[200];
    float cra;
};

int main()
{
    struct aluno a1 = {"Klaudio", 9.1};
    printf("%d %d\n", &a1.nome, &a1.cra);
}
```

Ponteiros (*pointers*)

- Os ponteiros são variáveis que têm como valores endereços de memória ou um valor especial, nil, utilizado para indicar que o ponteiro não pode ser usado para apontar para um endereço de memória.

Ponteiros (*pointers*)

- Os **ponteiros** são variáveis que têm como valores endereços de memória ou um valor especial, nil, utilizado para indicar que o ponteiro não pode ser usado para apontar para um endereço de memória.

- Ponteiros nos ajudam com o **armazenamento dinâmico** (malloc em C), bem como o **endereçamento indireto**.

Ponteiros (*pointers*)

```c
#include <stdio.h>

int main()
{
    int num = 28;
    int *numPtr = &num;
    // Endereço de memória de num e valor de numPtr
    printf("%d %d\n", &num, numPtr);
    // Valor de num e valor do endereço apontado por numPtr
    printf("%d %d\n", num, *numPtr);
}
```

Ponteiros (*pointers*)

- Em C, os ponteiros podem apontar para qualquer lugar da memória, não apenas para variáveis no monte (heap).

Ponteiros (*pointers*)

- Em C, os ponteiros podem apontar para qualquer lugar da memória, não apenas para variáveis no monte (heap).

- Um ponto problemático da linguagem é que pode-se até mesmo apontar para um local da memória que nem ao menos tem uma variável.

Ponteiros (*pointers*)

- Em C, os ponteiros podem apontar para qualquer lugar da memória, não apenas para variáveis no monte (heap).

- Um ponto problemático da linguagem é que pode-se até mesmo apontar para um local da memória que nem ao menos tem uma variável.

- Eles podem apontar para void * (valores de qualquer tipo), podendo-se fazer conversões p/ tipos específicos.

Valores de tipo de referência

- As variáveis de tipo de referência apontam, de fato, para um objeto ou valor na memória. Os ponteiros funcionam de forma semelhante, mas apontam para um endereço de memória.

Valores de tipo de referência

- As variáveis de tipo de referência apontam, de fato, para um objeto ou valor na memória. Os ponteiros funcionam de forma semelhante, mas apontam para um endereço de memória.

- Lembrete: Através de ponteiros, C simula passagens por referência.

Sistema de tipos

- A definição precisa de **associações entre tipo da variável, seus valores e operações possíveis** entre esses valores.

Sistema de tipos

- A definição precisa de **associações entre tipo da variável, seus valores e operações possíveis** entre esses valores.

- As **operações inadequadas entre tipos podem gerar erros** (e.g. multiplicar um boolean por uma string).

Sistema de tipos

- A definição precisa de **associações entre tipo da variável, seus valores e operações possíveis** entre esses valores.

- As **operações inadequadas entre tipos podem gerar erros** (e.g. multiplicar um boolean por uma string).

- As linguagens executam uma verificação de se os operandos e operadores são de tipos compatíveis.

Sistema de tipos

- Essas checagens e verificações servem para que não aconteça operações indevidas entre operandos e operadores.

Sistema de tipos

- Essas checagens e verificações servem para que não aconteça operações indevidas entre operandos e operadores.

- Podem ocorrer em tempo de compilação ou em tempo de execução.

Sistema de tipos

- Essas checagens e verificações servem para que não aconteça operações indevidas entre operandos e operadores.

- Podem ocorrer em tempo de compilação ou em tempo de execução.

- Pode ser que seja necessária a realização de uma conversão de tipo (automática) para garantir a compatibilidade (e.g. *casting*).

Sistema de tipos

- Uma linguagem pode ser estaticamente tipada ou dinamicamente tipada:

Sistema de tipos

- Uma linguagem pode ser **estaticamente tipada** ou **dinamicamente tipada**:
  - Nas **estaticamente tipadas**, as variáveis, parâmetros ou expressões têm tipos fixos, escolhido pelo programador ou inferido pelo processador da linguagem (e.g. C, Java)

Sistema de tipos

- Uma linguagem pode ser **estaticamente tipada** ou **dinamicamente tipada**:
  - Nas **estaticamente tipadas**, as variáveis, parâmetros ou expressões têm tipos fixos, escolhido pelo programador ou inferido pelo processador da linguagem (e.g. C, Java)
  - Nas **dinamicamente tipadas**, as variáveis, parâmetros e expressões podem assumir tipos diferentes durante a execução do programa. Apenas os valores possuem tipos fixos (e.g. Python, Ruby).

Sistema de tipos

```
public static void main(String[] args) {
    int num1 = 10; // explicitamente definida como "int"
    var num2 = 10; // implicitamente inferida como "int"
    System.out.println(num1 + num2);
}
```

Sistema de tipos

- As linguagens estaticamente tipadas possuem **melhor performance** em tempo de execução, bem como **maior segurança**.

Sistema de tipos

- As linguagens estaticamente tipadas possuem **melhor performance** em tempo de execução, bem como **maior segurança**.

- As linguagens dinamicamente tipadas possuem **maior flexibilidade**, se comparadas com linguagens estaticamente tipadas.

Sistema de tipos

- Uma linguagem pode ter tipagem forte ou tipagem fraca:

Sistema de tipos

- Uma linguagem pode ter tipagem forte ou tipagem fraca:
    - Nas com tipagem fraca, as conversões de tipo são, em geral, implícitas.

Sistema de tipos

- Uma linguagem pode ter **tipagem forte** ou **tipagem fraca**:
  - Nas com **tipagem fraca**, as conversões de tipo são, em geral, implícitas.
  - Nas com **tipagem forte**, as conversões de tipo são, em geral, explícitas. Os erros de tipo são sempre detectados, seja em tempo de compilação, seja em tempo de execução.

Sistema de tipos

- Podemos verificar equivalência de tipos na hora de fazer as checagens de tipo. Os tipos podem ter **equivalência estrutural** ou **de nome**.

Sistema de tipos

- Podemos verificar equivalência de tipos na hora de fazer as checagens de tipo. Os tipos podem ter **equivalência estrutural** ou **de nome**.
    - Na **equivalência estrutural**, os tipos são equivalentes se possuem o mesmo conjunto de valores.

Sistema de tipos

- Podemos verificar equivalência de tipos na hora de fazer as checagens de tipo. Os tipos podem ter **equivalência estrutural** ou **de nome**.
  - Na **equivalência estrutural**, os tipos são equivalentes se possuem o mesmo conjunto de valores.
  - Na **equivalência de nomes**, os tipos são equivalentes se ambos foram definidos na mesma declaração.

Sistema de tipos

- Observe o exemplo de código abaixo:

```
struct Posicao {int x, y;);
struct Data {int d, m;};
struct Posicao p;
struct Data d;
void mostrar(struct Data d);
mostrar(d);
mostrar(p);
```

- mostrar(d) funciona se a linguagem adota equivalência estrutural ou nomes. mostrar(p) só se a linguagem adotar equivalência estrutural.

Sistema de tipos

- As linguagens podem adotar valores que são ditos de primeira classe ou segunda classe.

Sistema de tipos

- As linguagens podem adotar valores que são ditos de **primeira classe** ou **segunda classe**.
    - Os **valores de primeira classe** podem ser utilizados de todas as maneiras possíveis (e.g. passados como argumento, avaliados, atribuídos, fazerem parte de tipos compostos)

Sistema de tipos

- As linguagens podem adotar valores que são ditos de **primeira classe** ou **segunda classe**.
  - Os **valores de primeira classe** podem ser utilizados de todas as maneiras possíveis (e.g. passados como argumento, avaliados, atribuídos, fazerem parte de tipos compostos)
  - Os **valores de segunda classe** não podem ser utilizados de todas as maneiras possíveis.

Sistema de tipos

```
filtra = lambda f, l: [elem for elem in l if f(elem)]
eh_par = lambda x: x % 2 == 0

print(filtra(eh_par, range(0, 11)))
```

Expressões

- Expressões são trechos de programas que podem ser avaliados, resultando um valor.

Expressões

- **Expressões** são trechos de programas que podem ser avaliados, resultando um valor.

- As expressões **podem ter um tipo**, que é o tipo do valor resultado pela avaliação da expressão.

Expressões

- **Expressões** são trechos de programas que podem ser avaliados, resultando um valor.

- As expressões **podem ter um tipo**, que é o tipo do valor resultado pela avaliação da expressão.

- Existem **diferentes tipos de expressão**, listados a seguir:

Tipos de expressão

- Literais: expressões que possuem um valor fixo, sendo este valor de algum tipo (e.g. ‘a’, 10, “Klaus”).

Tipos de expressão

- Literais: expressões que possuem um valor fixo, sendo este valor de algum tipo (e.g. ‘a’, 10, “Klaus”).

- Agregados: expressões que constroem valores compostos a partir de seus componentes (e.g. {“Klaus”, 32}).

Tipos de expressão

- Literais: expressões que possuem um valor fixo, sendo este valor de algum tipo (e.g. ‘a’, 10, “Klaus”).

- Agregados: expressões que constroem valores compostos a partir de seus componentes (e.g. {“Klaus”, 32}).

- Chamadas de função: expressões que computa valores aplicando funções a argumentos (e.g. fun(x) - x).

Expressões

- Expressões condicionais: expressão formada por subexpressões, sendo que uma delas é escolhida e avaliada (e.g. if x > y then x else y em ML). Algumas linguagens recorrem a comandos condicionais.

Expressões

- Expressões condicionais: expressão formada por subexpressões, sendo que uma delas é escolhida e avaliada (e.g. if x > y then x else y em ML). Algumas linguagens recorrem a comandos condicionais.

- Expressões iterativas: expressão que contém computações sobre uma série de valores, resultando um valor (e.g. *list comprehension* em Python)

Expressões

- **Acessos a constantes e variáveis**: expressões que contém referências a constantes e variáveis nomeadas (e.g. final float PI = 3.14; float r;, circ = 2 * PI * r;)

Tipo abstrato de dado

- Os tipos abstratos de dados são tipos de dados definidos pelo usuário, suportados por muitas linguagens de programação. Estes serão vistos mais adiante.

Tipo abstrato de dado

- Os **tipos abstratos de dados** são tipos de dados definidos pelo usuário, suportados por muitas linguagens de programação. Estes serão vistos mais adiante.

- Alguns recursos poderosos, como sobrecarga (*overload*), polimorfismo e subtipos surgiram em linguagens mais recentes, a contraponto de linguagens clássicas, que normalmente não têm esses recursos.

Referências

SEBESTA, Robert M. **Conceitos de Linguagens de Programação**. Bookman, 2018.